1. Describe the structure of a YOLO dataset.

A YOLO dataset contains images, corresponding annotation files (txt), and a YAML configuration file that tells YOLO how to load the dataset.

2. Explain the YOLO label format and why coordinates are normalised.

YOLO uses a text file to store object annotations. Each line represents one object in an image and contains five values:

- class_id is the integer identifying the object class
- x_center is the horizontal center of the bounding box
- y_center is the vertical center of the bounding box
- width is the width of the bounding box
- height is the height of the bounding box

The four coordinates are normalised to values between 0 and 1, relative to the image dimensions. Normalisation makes bounding boxes independent of the original image resolution. The same relative box can be represented consistently for images of different sizes, making the dataset easier to process during training.

3. What is a pretrained model, and why can it perform reasonably well before training?

A pretrained model is a model that has already learned useful features from a large dataset before being applied to a new task.

In my lab06, because elephants and zebras are already included in COCO, the pretrained model can often detect them reasonably well without additional training on the custom dataset. When I fine-tune the model on my custom dataset, it adjusts its weights to improve performance on the specific training data.

4. What do IoU, Precision, Recall, and mAP measure in object detection?

- IoU measures how closely a predicted bounding box overlaps with the ground-truth bounding box. IoU ranges from 0 to 1. A value of 1 means the boxes overlap perfectly. IoU is also used to determine whether a detection counts as a true positive at a specified overlap threshold.
  $IoU = \frac{Intersection\ Area}{Union\ Area}$

- Precision measures how many predicted detections are correct.
  $Precision = \frac{True\ Positive}{True\ Positive + False\ Positive}$

- Recall measures how many actual objects the model successfully detects.
  $Recall = \frac{True\ Positive}{True\ Positive + False\ Negative}$
  Precision focuses on the correctness of predictions, while recall focuses on finding the actual objects.

- mAP evaluates the overall detection performance by combining Average Precision (AP) across the object classes. AP summarises the precision-recall relationship across different confidence thresholds. mAP then averages the AP values across the classes being evaluated.
  $mAP = \frac{AP_{class\ 1} + AP_{class\ 2} + \ldots + AP_{class\ n}}{n}$

5. What is the difference between mAP50 and mAP50–95?

- mAP50: A prediction is eligible to count as a correct detection when its IoU with the ground-truth box is at least 0.50.
- mAP50–95: The model is evaluated at ten IoU thresholds from 0.50 to 0.95, in increments of 0.05. The AP values are averaged across these thresholds and classes. Because higher thresholds require tighter box alignment.
  ⇒ This metric also measures how accurately the model localises objects.

6. Compare the pretrained and fine-tuned model results, and explain why performance improved or decreased.

Metric	Pre-trained	Fine-tuned
Precision	90.70%	73.27%
Recall	86.67%	92.78%
mAP50	96.85%	89.65%
mAP50-95	84.04%	73.20%

Table 1 - Overall performance comparison

Class	Metric	Pre-trained	Fine-tuned
Elephant	AP50	94.21%	79.80%
Elephant	AP50-95	78.53%	56.86%
Zebra	AP50	99.50%	99.50%
Zebra	AP50-95	89.55%	89.55%

Table 2 - Per-class performance comparison

The difference in Zebra detection is not obvious because there are not much images with zebra in the dataset

Although fine-tuning increased recall by 6.11 percentage points, precision decreased by 17.43 percentage points. This indicates that the fine-tuned model detected more of the annotated objects but also produced a greater proportion of incorrect detections.

Overall, the Fine-tuned model performed worse because of these reasons:

- The pretrained model already knows these animals. YOLO11n's COCO weights have already learned to recognise elephants and zebras, so fine-tuning on a relatively small custom dataset may not add useful information.
- Fine-tuning may have altered useful features. Training for 30 epochs could have changed some previously useful weights without improving generalisation.

Below are the metrics and results of Fine-tuned and Pre-trained models after testing:

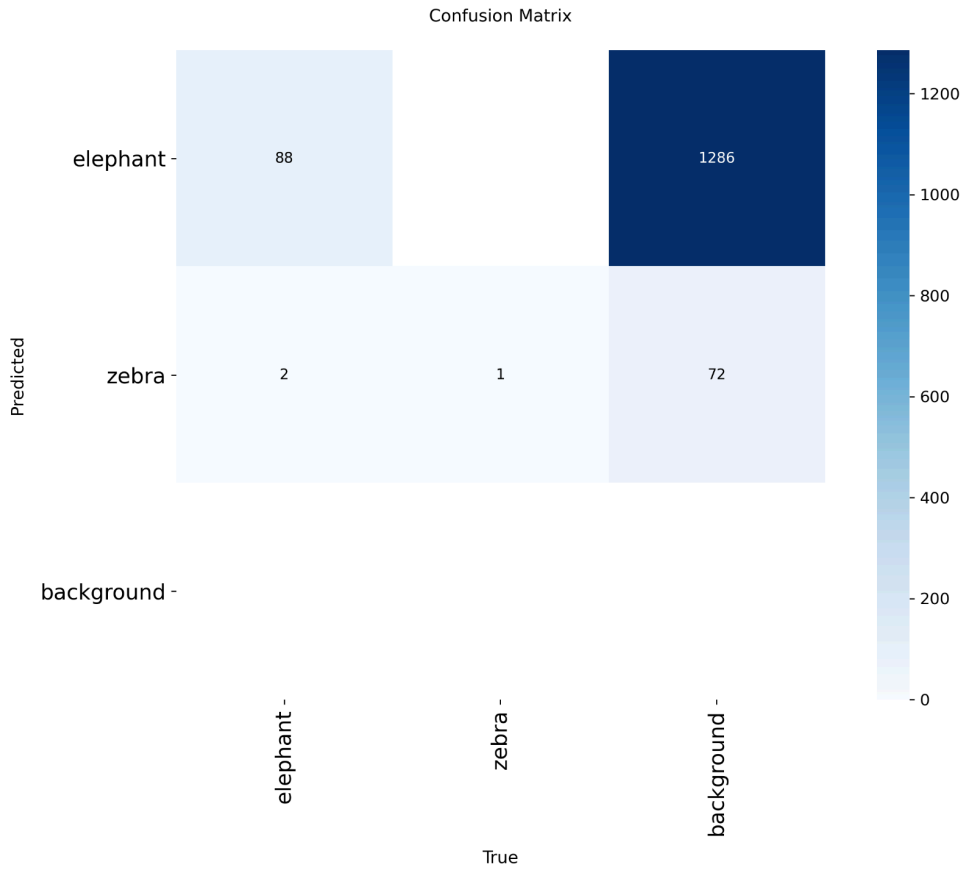


Figure 1 - Confusion matrix for Fine-tuned model

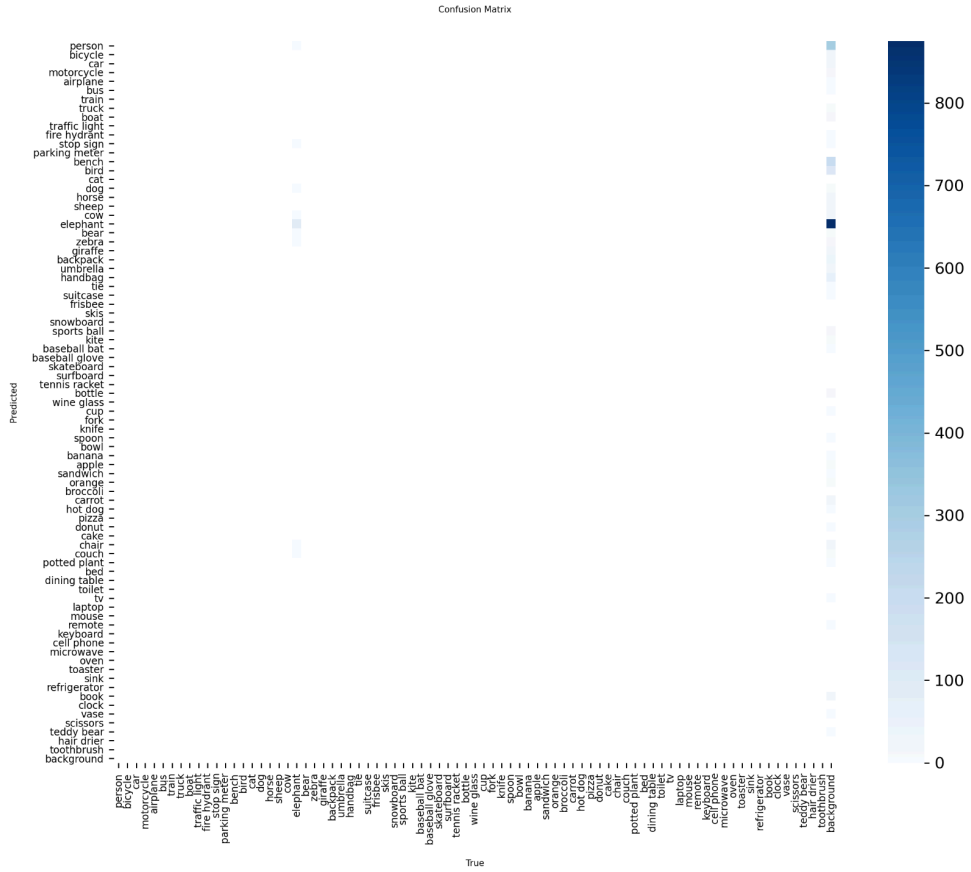


Figure 2 - Confusion matrix for Pre-trained model

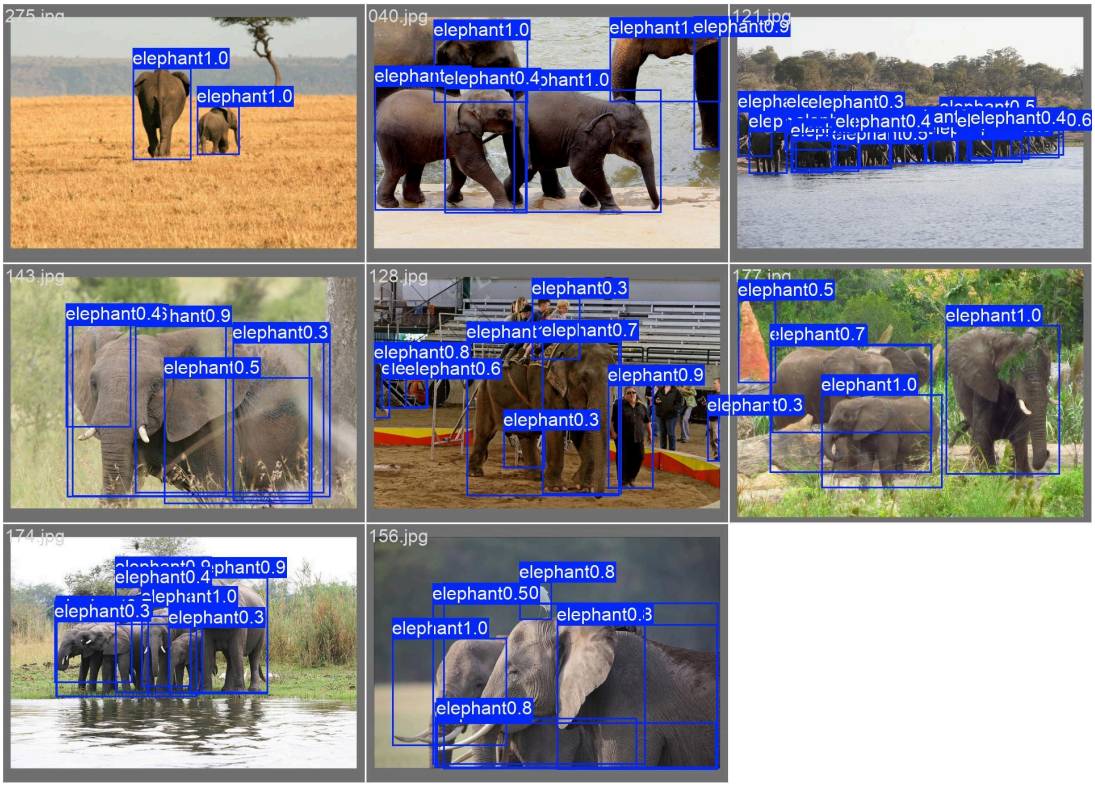


Figure 3 - Fine-tuned model's prediction on val_batch2

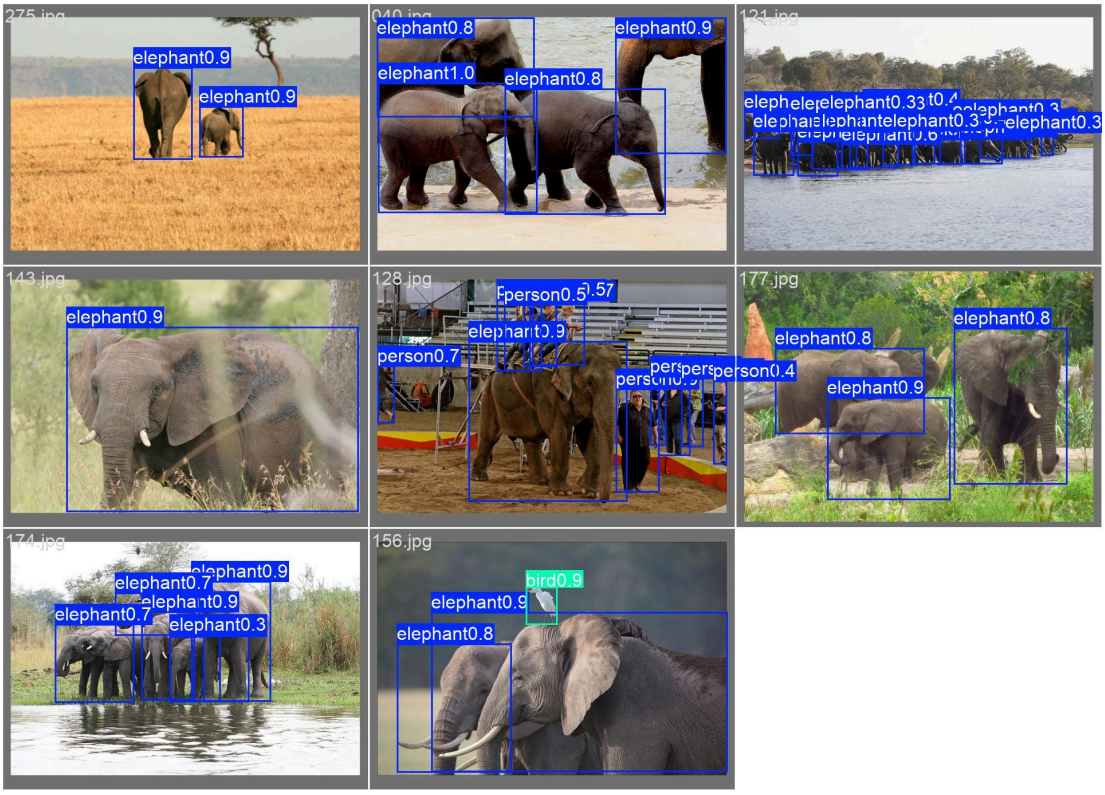


Figure 4 - Pre-trained model's prediction on val_batch2

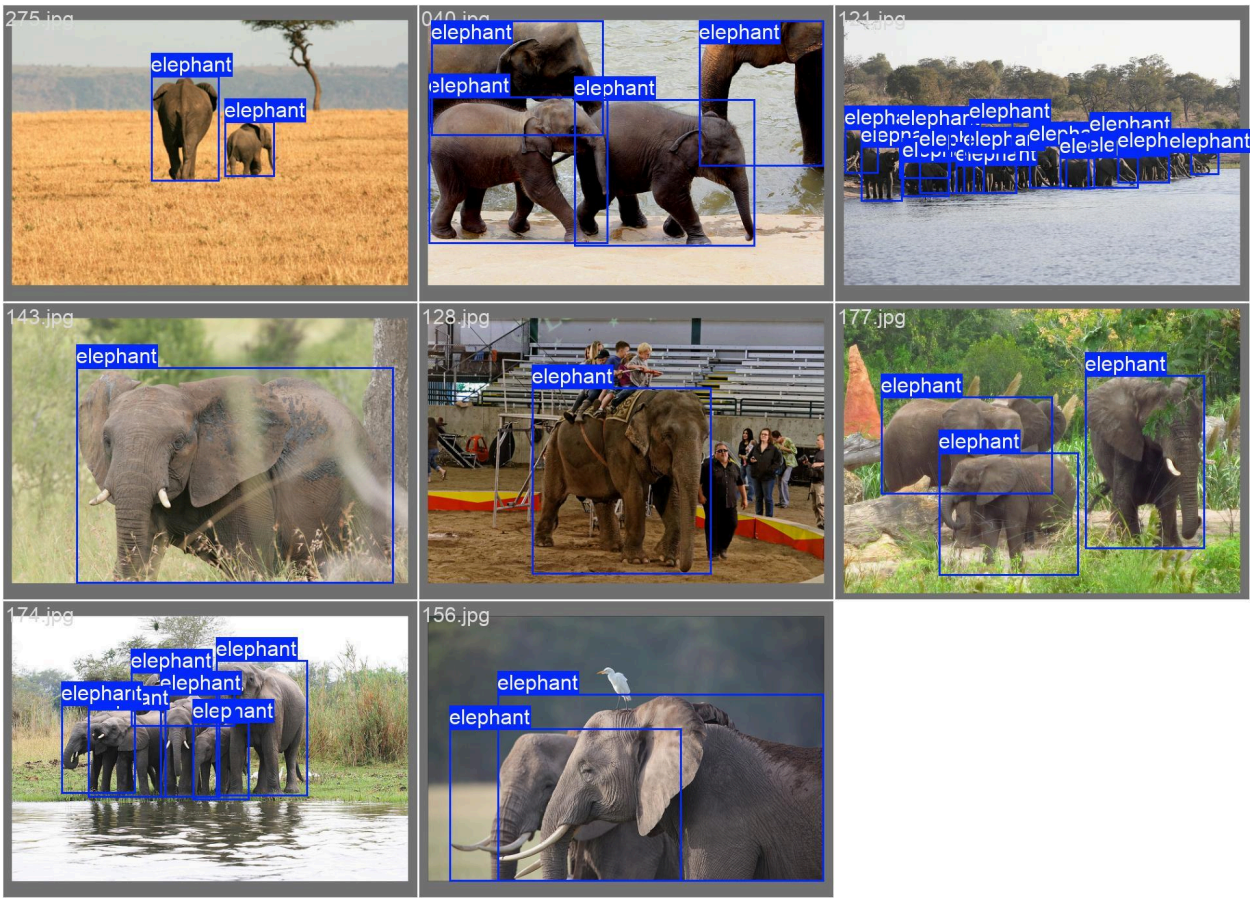


Figure 5 - val_batch2 ground-truth